PyTorch 기반 ANN 코드 이해

ANN의 핵심 구조와 학습 원리 중심으로 설명

정승한/기계공학과

ANN · PyTorch · 학습 루프

2026/05/12

목차

01 ANN에 대한 설명

02 ANN의 특징

03 핵심 코드 설명

04 질문으로 정리

ANN에 대한 간단한 설명

1. 인간의 뉴런을 기반으로

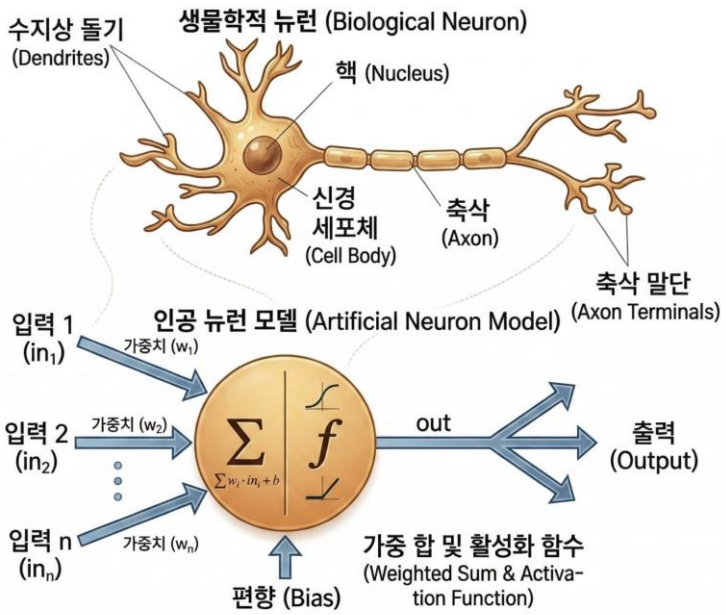
ANN은 Artificial Neural Network, 즉 인공신경망
여러 개의 계산 단위를 층처럼 연결
입력과 출력 사이의 관계를 학습하는 AI모델

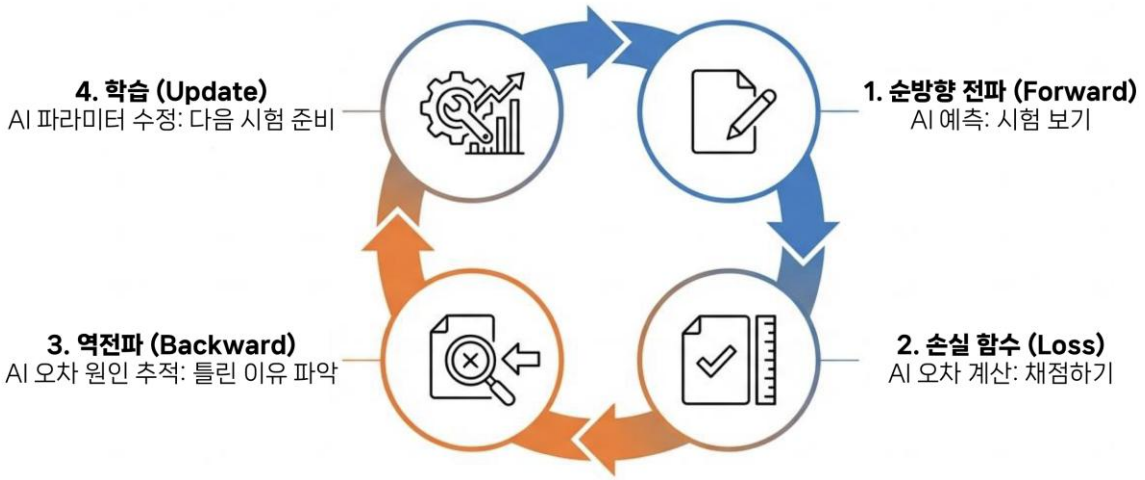
2. 여러 개의 층이 존재

입력층, 은닉층, 출력층과 같은 여러 개의 층과
이를 연결하는 가중치, 편향이 존재

3. ANN 학습 방식

입력 -> 순전파 -> 손실 함수 계산 -> 역전파 -> Update





ANN의 핵심 특징

1. 데이터 기반 학습

정확한 공식을 사람이 만들지 않아도, 입력 X와 정답 y의 짝을 보고 관계를 학습한다.

2. 비선형 표현

Linear 계산만 있으면 단순하지만, ReLU 같은 활성화 함수가 들어가면 복잡한 곡선을 표현할 수 있다.

3. 반복적 개선

예측값과 정답의 차이인 Loss를 줄이는 방향으로 가중치와 편향을 계속 수정한다.

ANN은 데이터 속 패턴을 보고 입력과 출력 사이의 관계를 근사하는 모델이다.

핵심 코드

```
import torch
import torch.nn as nn
import torch.optim as optim

X = torch.tensor(X_np, dtype=torch.float32)
y = torch.tensor(y_np, dtype=torch.float32)

class SimplePredictiveAI(nn.Module):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(...)

    def forward(self, x):
        return self.network(x)

model = SimplePredictiveAI()
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

데이터를 Tensor로 변환한다.

신경망 구조를 class 안에 정의한다.

Loss는 예측이 얼마나 틀렸는지 계산한다.

Optimizer는 Loss를 줄이도록 가중치를 업데이트한다.

데이터와 Tensor 변환

PyTorch 모델이 학습할 수 있는 자료형으로 바꾼다

```
X_np = np.linspace(-3,3,200).reshape(-1,1)
```

-3부터 3까지 입력값 200개 생성. reshape(-1,1)은 "입력 변수 1개짜리 데이터 200개" 형태로 바꾼다는 뜻.

```
y_np = sin + cos + 선형항 + noise
```

예제에서는 실제 데이터 대신 사람이 만든 정답 y를 사용. 복잡한 비선형 함수를 흉내 낸 것.

```
X = torch.tensor(X_np, dtype=torch.float32)
```

NumPy 배열을 PyTorch Tensor로 변환. 모델 계산, 자동미분, GPU 연산을 위해 필요.

```
y = torch.tensor(y_np, dtype=torch.float32)
```

정답도 Tensor로 변환. 예측값 y_pred와 비교해 Loss를 계산할 수 있게 한다.

모델 정의: __init__와 forward

__init__은 부품을 조립하고, forward는 데이터를 통과시킨다

```
class SimplePredictiveAI(nn.Module):
    def __init__(self):
        super().__init__()
        self.network = nn.Sequential(
            nn.Linear(1, 64),
            nn.ReLU(),
            nn.Linear(64, 1)
        )

    def forward(self, x):
        return self.network(x)
```

class SimplePredictiveAI(nn.Module)

PyTorch 신경망 모델 설계도를 만든다. nn.Module을 상속받아야 PyTorch 모델로 인식된다.

def __init__(self)

모델이 처음 생성될 때 실행된다. Linear, ReLU 같은 신경망 층을 이곳에서 미리 등록한다.

def forward(self, x)

입력 x가 들어왔을 때 어떤 순서로 계산할지 정의한다. model(X)를 실행하면 내부적으로 forward가 호출된다.

중요한 구분

층은 __init__에서 한 번 만들고, forward에서는 그 층에 데이터를 통과시킨다.

Linear와 활성화 함수

입력을 변환하고, 비선형성을 넣어 ANN답게 만든다

```
nn.Linear(1, 64)
nn.ReLU()
nn.Linear(64, 64)
nn.ReLU()
nn.Linear(64, 1)
```

nn.Linear(1,64)

입력값 1개를 받아 내부 특징 64개로 변환하는 선형층이다. 내부적으로 W·x + b 계산을 한다.

ReLU

0보다 작은 값은 0, 큰 값은 그대로 통과시킨다. 비선형성을 넣어 복잡한 패턴을 학습하게 한다.

Sigmoid / tanh / Leaky ReLU

Sigmoid는 0~1 확률 출력에 자주 쓰고, tanh는 -1~1 범위, Leaky ReLU는 음수도 조금 통과시킨다.

Loss와 Optimizer

틀린 정도를 계산하고, 그 값을 줄이도록 모델을 수정한다

```
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.01)
```

MSELoss

회귀 문제에서 많이 쓰는 손실 함수. 예측값과 정답값의 차이를 제곱해 평균낸다.

BCELoss

성공/실패, 정상/불량처럼 정답이 0 또는 1인 이진 분류에 사용한다. 출력이 확률값이어야 한다.

Adam

Optimizer의 한 종류. gradient를 이용해 각 가중치를 어떻게 바꿀지 결정한다. 비교적 빠르고 안정적으로 많이 사용된다.



학습 루프 핵심 코드

예측 → 오차 계산 → 역전파 → 업데이트를 반복한다

```
for epoch in range(epochs):
    model.train()
    y_pred = model(X)
    loss = criterion(y_pred, y)

    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
```

model.train()

학습 모드로 전환. Dropout, BatchNorm이 있을 때 작동 방식이 달라진다.

y_pred = model(X)

현재 가중치로 입력 X에 대한 예측값을 만든다.

loss.backward()

Loss를 줄이기 위한 각 파라미터의 gradient를 계산한다.

optimizer.step()

Adam이 gradient를 이용해 가중치와 편향을 실제로 업데이트한다.

실제 데이터로 바꾸면 어떻게 되나?

```
X_np = np.array([
  [200, 60, 20, -0.02],
  [205, 60, 25,  0.00],
  [195, 55, 30,  0.05],
])

y_np = np.array([[1], [1], [0]])
# 1 = 정상, 0 = 불량
```

입력 X

노즐 온도, 베드 온도, 첫 레이어 속도, Z-offset처럼 측정 또는 설정 가능한 변수들.

정답 y

각 조건에서 실제 첫 레이어가 정상인지 불량인지 기록한 결과. 지도학습에서는 y가 필요하다.

모델 첫 층 변경

입력 변수가 4개이면 첫 Linear 층은 nn.Linear(4,64)가 된다.

공부 질문 정리: AI 관련 핵심 개념

시드(seed)는 왜 고정하나?

랜덤 노이즈와 초기 가중치를 같게 만들어 실험을 재현 가능하게 한다.

torch.tensor는 왜 쓰나?

PyTorch 모델 계산, 자동미분, GPU 연산을 위해 Tensor 형태가 필요하다.

__init__은 왜 층을 정의하나?

모델 생성 시 Linear, ReLU 같은 부품을 한 번 등록하기 위해서다.

BCELoss는 언제 쓰나?

정상/불량처럼 0 또는 1을 예측하는 이진 분류 문제에 쓴다.

Optimizer와 Adam은?

Optimizer는 가중치 수정 방법이고, Adam은 자주 쓰이는 안정적인 optimizer다
.

Dropout과 BatchNorm은?

Dropout은 과적합 완화, BatchNorm은 학습 안정화를 돕는다.

마무리

데이터 준비

X와 y를 Tensor로 변환

모델 정의

Linear + 활성화 함수로 구조 구성

손실 계산

예측값과 정답의 차이 계산

가중치 갱신

Adam이 Loss를 줄이는 방향으로 수정

복잡한 비선형 데이터를 ANN이 반복 학습을 통해 근사하는 과정을 보여준다. 실제 프로젝트에서는 수식 대신 실험 데이터의 X–y 짝을 준비해 같은 학습 흐름으로 적용할 수 있다.